Hermes

See also: Glossary for acronyms/technical terms used below.

A self-hosted AI agent that runs entirely on your own hardware — no cloud API key, no data sent to a third party — reachable from Telegram, and built to be studied and adapted, not only deployed as-is.

What this actually is

- **A working example, not just a demo.** Clone it, follow one guide, and you have a real AI agent answering you on Telegram within an hour, running a real local model on your own machine or a rented server.
- **A base for learning how the pieces fit together.** Agent orchestration (Hermes Agent), local model serving (llama.cpp + llama-swap), and — as specced work in this repo progresses — retrieval-augmented generation (RAG), model evaluation, and multi-channel routing. Every design decision in this repo is written down with its reasoning (see `docs/ARCHITECTURE.md` and the linked issues below), not just the code.
- **A starting point to adapt, not a fixed product.** Swap the model, add your own documents to search over, wire up a different channel, scale it to more than one user — the two reference deployments below are meant to be forked and changed, for a personal assistant or an enterprise deployment alike.

Who this is for

- Someone who wants a working AI agent without sending their data to a cloud provider.
- Someone learning how an agent, a local model, and a channel (Telegram today) actually connect in a real, running system — not just in theory.
- Someone who wants a documented starting point to adapt: a different model, a company's own documents (RAG), a different channel, more than one user.

How it's built

Hermes Agent (Nous Research's open-source, self-hosted AI agent) is wired to **local** LLMs served by llama.cpp — no external API key, no data sent to a third-party service, everything runs on hardware you own or rent. llama-swap sits between them so you can list more than one model and switch between them from inside Hermes, rather than being locked to whatever was configured at install time.

Two complete configurations, independent from each other, **each with both a Docker path and a native (no-Docker-at-all) path** for every component:

Configuration	Where	Model serving	Hermes	Guide
macOS ARM64	An Apple Silicon Mac	native (Metal acceleration) — same either way	Docker (`linux/arm64`) *or* native	`macos-arm64/`
Linux x86-64	A rented VPS	Docker (CPU) *or* native	Docker (`linux/amd64`) *or*	`linux-x86_64-`

Configuration	Where	Model serving	Hermes	Guide
			native	vps/

Both wire Hermes to Telegram (see shared/telegram-setup.md), ship one model by default — **Meta-Llama-3.1-8B-Instruct** in Q4_K_M (see shared/model-notes.md, and shared/managing-models.md to add more) — and default to official **prebuilt binaries** rather than building anything from source (see shared/prebuilt-binaries.md). The Hermes web dashboard, when enabled, requires a real username/password (it refuses to start without one — see each platform's README). Both also ship ghcr.io/ka8t/hermes — the same Hermes, plus two bundled skills that let you describe a new agent in plain language and get a working profile back (see shared/managing-models.md and skills/agent-creation/), and an enterprise-safe approval default so nothing destructive ever runs without an explicit human answer (see shared/enterprise-safety.md).

How it works

Three separate pieces, each with one job, connected through Hermes Agent at the center:

Hermes Agent is the orchestrator. It holds your conversation memory, knows a set of tasks it can perform (its "skills"), and decides what to do with every message it receives — but on its own it can neither hear nor speak, nor think. It needs two things plugged into it: a gateway to receive/send human messages, and a brain (an LLM) to generate its replies and decide which tools to call.

Telegram is the gateway used here — the channel that lets a human talk to Hermes from their phone, rather than needing an SSH/terminal connection to the machine running it. Hermes Agent supports several gateways (Discord, Slack, email...); Telegram was chosen for this project because it's the fastest to set up (one bot token from BotFather, one allow-list of user IDs).

llama.cpp (via llama-swap in front of it) is the local brain. It's the engine that actually runs the language model (loads the GGUF weights, generates tokens) and exposes it over an OpenAI-compatible HTTP API. Hermes Agent sends it a request every time it needs to "think" — to reply, to decide whether to use a tool, or to follow a skill.

The full path a message takes:

```
Phone (Telegram)
  → Telegram Bot API
    → Hermes Agent's Telegram gateway
      → Hermes's agent loop (assembles system prompt + skills + tools +
history)
        → HTTP request to llama-swap
          → llama-server (llama.cpp) generates the reply
        ← reply (plain text, or a structured tool call)
      ← Hermes runs the tool if needed, updates its memory
    ← Telegram gateway sends the final reply
  ← Telegram Bot API
Phone (Telegram)
```

Why native on Mac but Docker by default on the VPS?

Docker Desktop for Mac cannot expose the Metal GPU to a container — running `llama-server` inside it would fall back to CPU-only inference. On a Mac, llama-swap and the `llama-server` it spawns therefore always run natively (full Metal access), while Hermes itself can go either way (Docker by default; native is documented too, see each platform's README). On a regular Linux VPS (no dedicated GPU), that distinction doesn't apply — Docker Compose is the simpler default for everything, model serving included — but a fully native path (no Docker anywhere) is documented as an alternative for both platforms, for anyone who'd rather not run Docker at all.

Why these defaults?

Hermes requires at least 64,000 tokens of context to work properly (memory + tools + history are sent on every call), and tool calls only execute with llama.cpp's `--jinja` flag — both easy-to-miss points are explained and already handled in both configurations. See `shared/model-notes.md`. Every binary this repo runs is fetched prebuilt rather than compiled, which means trusting each project's CI pipeline rather than just its source — a trade-off made deliberately, not overlooked; see the "Trust considerations" section of `shared/prebuilt-binaries.md`.

Quick start

```
git clone https://github.com/ka8t/Hermes.git
cd Hermes

# On an Apple Silicon Mac
cd macos-arm64 && cat README.md

# On a Linux x86-64 VPS
cd linux-x86_64-vps && cat README.md
```

Repository structure

```
Hermes/
├── macos-arm64/          # native llama-swap + llama.cpp (Metal); Hermes
in Docker or native
│   └── scripts/            # download/run scripts for both llama.cpp and
native Hermes
├── linux-x86_64-vps/     # llama-swap, llama.cpp and Hermes: Docker
Compose or fully native
│   └── scripts/            # download/run scripts for both llama.cpp and
native Hermes
├── docker/               # ghcr.io/ka8t/hermes — Hermes + bundled skills +
safe defaults
├── skills/agent-creation/  # the guided agent-creation skills + starter
templates
├── eval/                 # model/hardware evaluation: BFCL + llama-bench
(see shared/model-evaluation.md)
├── docs/ARCHITECTURE.md  # living system architecture doc, diagrams — kept
current on every change
```

```
└── shared/                # platform-independent docs, linked from both
guides
```

- docs/ARCHITECTURE.md — components, deployment topologies, message flow, and what's specced vs. live
- docs/GLOSSARY.md — acronyms and technical terms used across this repo's docs
- shared/telegram-setup.md — bot creation, environment variables
- shared/model-notes.md — GGUF model choice, context constraints
- shared/managing-models.md — add / switch / remove models via llama-swap
- shared/prebuilt-binaries.md — official binaries used, per platform
- shared/enterprise-safety.md — the approvals default, and what it doesn't cover
- shared/multi-user-agents.md — one Hermes profile per user, onboarding, current platform coverage
- shared/cloudflare-tunnel-setup.md — public HTTPS exposure for WhatsApp/Teams (unverified — see #16)
- shared/whatsapp-setup.md — WhatsApp Business Cloud API channel (unverified)
- shared/teams-setup.md — Microsoft Teams channel (unverified)
- shared/hardware-sizing.md — check your real CPU/RAM/GPU before trusting this repo's documented performance numbers
- shared/gpu-setup.md — optional NVIDIA/AMD/Intel GPU support for the Linux VPS path (implemented, not live-verified — see #13)
- shared/model-evaluation.md — BFCL (tool-calling reliability, eval/) and llama-bench (throughput, specced only) evaluation (see #28). Real BFCL v4 scores for the default model on this Mac (M1, Metal): **54.75%** on simple_python (400/400 cases), **52.50%** on parallel (200/200) — multi_turn categories not yet completed, see shared/model-evaluation.md for why.
- shared/model-comparison.md — /compare, an in-channel command to ask several models the same question side by side (specced, not yet implemented — see #39)

Security

No secrets are committed: .env is git-ignored (only .env.example files are tracked), as is any .gguf model (too large) and Hermes's persistent state (data/, including memory, sessions, and the generated config.yaml / models.yaml). See .gitignore. Back up data/ with hermes backup before anything risky — see each platform's "Common operations".

Sources

- Hermes Agent — official repository: https://github.com/NousResearch/hermes-agent
- Official documentation: https://hermes-agent.nousresearch.com/docs/
- llama.cpp — official repository: https://github.com/ggml-org/llama.cpp
- llama-swap — official repository: https://github.com/mostlygeek/llama-swap